



3

MVVM for Maintainability and Testability

When building XAML-based applications, one of the most important design patterns to learn is the MVVM pattern. MVVM provides a clear separation of concerns between the XAML markup in the view and the C# code in the View Model using data binding. With this separation comes ease of maintenance and testability. The View Model classes can be tested without taking a dependency on the underlying **User Interface (UI)** platform. For large teams, another benefit of this separation is that changing the XAML enables UI designers to work on the UI independently of developers who specialize in writing the business logic and the backend of the application.

In this chapter, you will learn about the following concepts:

- Fundamentals of the MVVM design pattern
- Popular MVVM frameworks
- Implementing MVVM in WinUI applications
- Handling View Model changes in the view
- Event handling in MVVM
- Streamline View Model implementations with the MVVM Toolkit

By the end of the chapter, you will understand the basics of the MVVM design pattern, have some familiarity with some popular MVVM frameworks available to developers, and know how to implement

MVVM in a WinUI application. We will wrap up by getting hands-on with the open source **MVVM Toolkit**.

Technical requirements

To follow along with the examples in this chapter, please refer to the *Technical requirements* section of [***Chapter 2**, Configuring the Development Environment and Creating the Project*](#).

You will find the code files of this chapter here:

<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter03>

Understanding MVVM

MVVM was introduced by Microsoft in 2005 and gained popularity with developers following the launch of **Windows Presentation Foundation (WPF)** and **Silverlight** because it lends itself so well to building applications with XAML. It is like the **Presentation Model** pattern, which was created by Martin Fowler – one of the most influential proponents of design patterns.

The MVVM pattern consists of the following three layers:

- **Model:** The Model layer contains the application's business logic and should perform all the data access operations. The View Model communicates with the Model to retrieve and save the application's data.
- **View:** The View layer is only responsible for the presentation of data in the application. The layout or structure is defined here, along with style definitions. This is the layer responsible for interacting with the user and receiving input events and data. The View is aware of the View Model only through data-binding expressions.

- **ViewModel:** The View Model (or ViewModel) layer is responsible for maintaining the state of data for the View. It has a set of properties that provide data to the View through data binding and a set of commands invoked by the View in response to user input events. View Model classes have no knowledge of their corresponding Views.

MVVM – the big picture

Let's look at how the components of MVVM fit into the overall architecture of an application implementing the pattern, depicted in the following diagram:

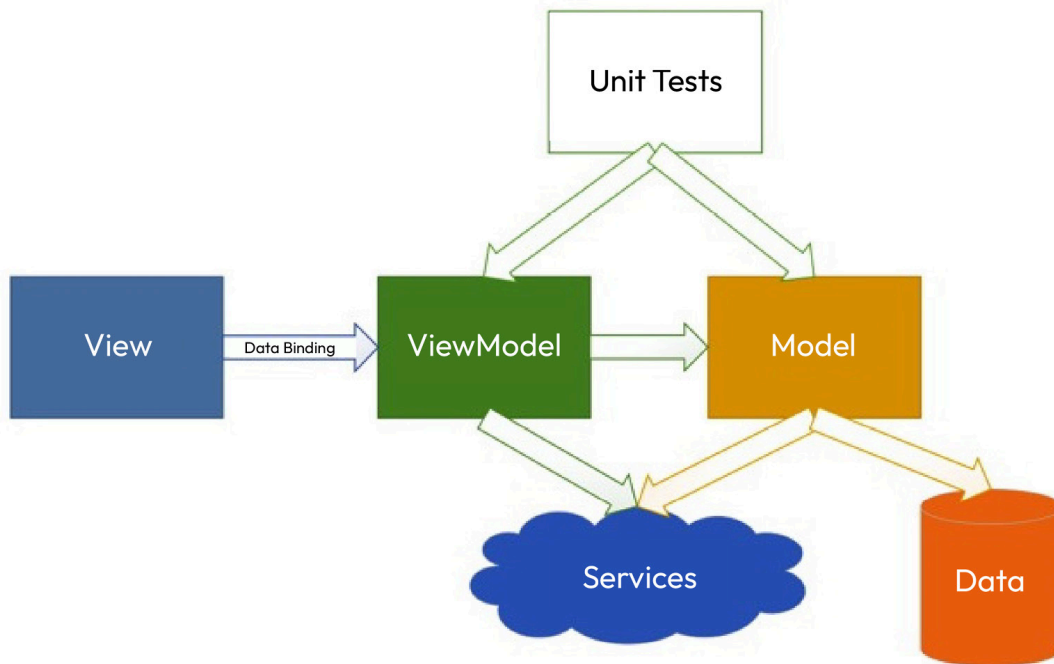


Figure 3.1 – The MVVM pattern in use

In *Figure 3.1*, you can see a representation of the MVVM pattern along with other parts of the application: services, data, and unit tests. The arrows in the diagram represent dependencies, not the data flow. You can imagine that data would need to move both ways along most of these pathways to create a functional application.

As with many design patterns, the MVVM pattern is meant to be a guide for developers to create reliable, maintainable applications.

However, not all developers implement the pattern in the same way. The differences usually lie in the implementation of the model. Some developers will create a domain model in the **Domain-Driven Design (DDD)** style. This makes sense for large applications with complex business logic. For simpler applications, the model may only be a simple data access layer, residing either on the client or behind a service layer. In some of these cases, the **Services** cloud in the preceding diagram would move between the View Model and Model layers.

The point of MVVM is to help guide you in building the best app for your users. As you gain experience with MVVM and WinUI, you will find the right implementation for your applications. A good way to get started is to find some frameworks that make it easier to work with MVVM.

MVVM libraries for WinUI

When working with MVVM in WinUI applications, you must create a little infrastructure code to facilitate data binding between Views and View Models. You can either write this yourself or choose a framework that abstracts this plumbing code away from your application. Although we will start by writing our own plumbing code for the application in the next section, let's review some popular MVVM frameworks for WinUI.

The MVVM Toolkit

We will discuss the **Windows Community Toolkit (WCT)** and **.NET Community Toolkit** in more detail in *[Chapter 9, Enhancing Applications with the Windows Community Toolkit](#)*, but one of the libraries included in this open source toolkit is the **MVVM Toolkit**. The toolkit supports any .NET application platform, including UWP, WinUI 3, .NET MAUI, Uno Platform, WPF, and others. You can get the **CommunityToolkit.Mvvm NuGet package** through the **NuGet Package**

Manager in Visual Studio or view its details on the NuGet website, at <https://www.nuget.org/packages/CommunityToolkit.Mvvm/>.

The library includes base classes to support **INotifyPropertyChanged**, **IMessenger**, and **ICommand**. It also includes other messaging and **Inversion of Control (IoC)** helper classes. In addition, the latest versions of the MVVM Toolkit use **Roslyn source generators** to reduce the amount of boilerplate code developers need to write.

NOTE

If you are unfamiliar with Roslyn, it is the code name of the .NET Compiler Platform. It was first released to manufacturing (RTM) with Visual Studio 2015, and it is the current compiler for the C# and VB.NET languages. Roslyn's extensibility, via the .NET Compiler Platform SDK, allows developers to create things such as custom source generators and source code analyzers for Visual Studio. For more information about the platform, you can check out the documentation on Microsoft Learn: <https://learn.microsoft.com/dotnet/csharp/roslyn-sdk/>.

Later in the chapter, after you have learned how to implement your own MVVM base class, we will leverage the MVVM Toolkit in our sample application.

The Prism Library

Prism started out as a library created and maintained by Microsoft. It was part of the **Patterns & Practices** guidance, reference architectures, and libraries that Microsoft's developer division used to maintain.

NOTE

The remaining Patterns & Practices projects still maintained by Microsoft are available on GitHub here: <https://github.com/mspnp>.

Microsoft decided to open source the Prism Library and transferred ownership to the community. The project is hosted on GitHub and can be found on the web at <https://prismlibrary.com/>. Prism has packages available for WPF, Xamarin, UWP, and WinUI projects.

Prism is much more than an MVVM framework. It also includes **application programming interfaces (APIs)** that help developers implement **dependency injection (DI)**, commands, and **EventAggregator** for loosely coupled application messaging. Prism can be added to a project through NuGet. There is also an installer available on the Prism site to add a Visual Studio project and item templates.

MVVMCross

MVVMCross is an MVVM framework that was first created for Xamarin developers. It now has NuGet packages available for Xamarin, .NET MAUI, WPF, UWP, and WinUI. As with Prism, MVVMCross does much more than facilitate data binding in WinUI applications. It has helpers for the following:

- Data binding
- Navigation
- Logging and tracing
- DI and IoC
- Unit testing

There are some additional libraries, but most are specific to .NET MAUI applications. The MVVMCross package can also be added to your project via NuGet. For more information about MVVMCross use with WinUI, check out their site: <https://www.mvvmcross.com/>.

Choosing a framework for WinUI applications

Using a third-party framework for building production XAML applications is a great choice. These provide built-in support for things such as logging and DI. For the WinUI application in this book, we will use the MVVM Toolkit for data binding at the end of this chapter.

However, first, we will implement the MVVM pattern from scratch. This will help you understand the underlying mechanisms of data binding, DI, and other core concepts relating to MVVM.

Before jumping into the code, you should have some background on exactly how data binding works in WinUI.

Understanding data binding in WinUI

In the previous chapter, you saw some simple examples of data binding, using both the **Binding** and **x:Bind** markup extensions. Let's dissect some of the components that allow the View to receive updates when the View Model data changes.

What are markup extensions?

An in-depth discussion of markup extensions is beyond the scope of this introductory book. In brief, they are a class that executes some logic to return a value to the XAML parser. You can identify their use in XAML by looking for some markup inside curly braces. Take this example of **Binding** in the **Text** property of a **TextBlock**:

```
<TextBlock Text="{Binding Path=Name, Mode=TwoWay}"/>
```

From this, you can derive that there is a markup extension class named **Binding** and that two of its properties are **Path** and **Mode**. This markup extension takes these properties, resolves a value, and returns it to the XAML parser for display in the application's View.

Some XAML markup languages allow developers to write their own custom markup extensions. WPF and .NET MAUI have custom markup extensions, but WinUI does not. If you're curious about the implementation of a markup extension, Microsoft Learn has documentation on how to create one for .NET MAUI:

<https://learn.microsoft.com/dotnet/maui/xaml/markup-extensions/create>.

Now, let's learn more about the **Binding** markup extension in WinUI.

Binding markup extension

As you briefly saw, the **Binding** markup extension maps data from the binding source, the View Model in MVVM, and provides it to a binding target in the View. These are all the properties of the **Binding** markup extension:

- **Path**: The path to the value in the data-binding source. For our application, this will be the property name on the View Model.
- **Converter**: If the data type of the source property does not match the data type of the control's property in the View, a **Converter** property is used to define the mapping between the two data types.
- **ConverterLanguage**: If a **Converter** property is specified, a **ConverterLanguage** property can optionally be set to support internationalization.
- **ConverterParameter**: If a **Converter** property takes a parameter, use the **ConverterParameter** property to provide it. It's not common to use a **Converter Parameter** property, and it is typically a **string** value. If you need to provide multiple values to a

Converter, you can concatenate them and then parse them inside the method.

- **ElementName**: This parameter is used when binding to an attribute of another element in the View.
- **FallBackValue**: If the data binding fails for any reason, you can specify a **FallBackValue** property to display in the View.
- **Mode**: This defines whether the data binding is **OneTime** (sets the value only when the XAML is first parsed), **OneWay** (fetches the value from the View Model when changes are detected), or **TwoWay** (the value flows both ways between the View and View Model). The default **Mode** setting for **Binding** varies depending on the control and property being bound. It is important to check the documentation if you are unsure of the default **Mode**.
- **RelativeSource**: This is used to define a data-binding source that is relative to the current control. This is usually used with control templates that get their data through a parent element.
- **Source**: Specifies the data-binding source. This is usually defined at a top-level control level in WinUI as the View Model. However, it is possible for child controls to set a different **Source**, overriding the **Source** inherited from their ancestor. The **Source** value defined at any level in the View will be inherited by all child elements unless a new **Source** is set.
- **TargetNullValue**: Specifies a default value to display if the data-binding source is resolved but has a **null** value.
- **UpdateSourceTrigger**: Specifies the frequency with which to update **TwoWay** binding sources. The options are **PropertyChanged**, **Explicit**, and **LostFocus**. The default frequency for most properties is **PropertyChanged**.

Path is the default property and is assumed when no property name is given for a parameter. For example, the earlier **TextBlock** example could also be written as follows:

```
<TextBlock Text="{Binding Name, Mode=TwoWay}"/>
```

Here, **Name** is assumed to be the value provided for the **Path** parameter. Providing two parameters without specifying parameter names will result in a XAML parser error.

The **Binding** markup extension is found in every XAML language. The other option for data binding, **x:Bind**, is not. It is only an option with UWP and WinUI.

x:Bind markup extension

x:Bind is an alternative markup extension for WinUI. It is faster and uses less memory than **Binding** and has better debugging support. It achieves this performance gain by generating code at compile time to be used during binding at runtime. By contrast, the **Binding** markup extension is executed by the XAML parser at runtime, which incurs additional overhead. Compile-time bindings also result in incorrect data binding expressions being caught when compiling, rather than generating data binding failures at runtime.

Another important distinction between **Binding** and **x:Bind** is that **Binding** requires a **Data Context** to be set. Data is bound to properties of objects within the **DataContext**. When using **x:Bind**, you are binding directly to properties on the current **Window** or **UserControl**. You can also bind events directly to event handlers in the code-behind for the Window with **x:Bind**.

While most of the properties of **x:Bind** are the same as those in **Binding**, let's highlight those that differ, as follows:

- **ElementName**: Not available in **x:Bind**. You must use **Binding** to data-bind to other XAML element attributes. If your application

must bind to other elements, **x:Bind** and **Binding** can be used in the same View.

- **Mode**: The only difference here is that the default **Mode** for **x:Bind** is usually **OneTime**, not **OneWay**.
- **RelativeSource**: Not available in **x:Bind**.
- **Source**: Not available in **x:Bind**. Instead, you will typically define a **ViewModel** property in the code-behind file of each View with a data type of the corresponding View Model class. You could also create a domain-specific name for the property, such as **MediaItems** for our application.
- **BindBack**: This property is unique to **x:Bind**. It allows a custom function to be called in **TwoWay** binding when reverse data binding is invoked. This is not commonly used, and we will not use it in our application.

x:Bind is a powerful and complex markup extension. For more information about it, you can read this page on Microsoft Learn:

<https://learn.microsoft.com/windows/uwp/xaml-platform/x-bind-markup-extension>.

Next, let's discuss **INotifyPropertyChanged**, the interface that enables changes in data-bound properties in the ViewModel to be reflected in the view.

Updating View data with INotifyPropertyChanged

So, how does the View get notified when data changes in the ViewModel? That magic lies within the **Microsoft.UI.Xaml.Data.INotifyPropertyChanged** interface. This interface consists of a single member, as shown here:

```
public interface INotifyPropertyChanged
```

```
{  
    event PropertyChangedEventHandler PropertyChanged;  
}
```

Every ViewModel class must implement this interface and raise the **PropertyChanged** event to update the View. Indicate which property is changed by passing its name in the **PropertyChangedEventArgs** parameter. To refresh all properties, pass **null** or **string.Empty** as the property name, but be aware of the potential performance implications this can have on large views.

Updating collection data with **INotifyCollectionChanged**

INotifyPropertyChanged works great for most properties, but it will not update the View if items from a collection have been added or removed. This is where the **Microsoft.UI.Xaml.Interop.INotifyCollectionChanged** interface is used. Again, this interface only has a single member, as shown here:

```
public interface INotifyCollectionChanged  
{  
    event NotifyCollectionChangedEventHandler  
        CollectionChanged;  
}
```

None of the collections or collection interfaces commonly used in .NET (**List<T>**, **IEnumerable<T>**, and so on) implement this interface. You can create your own collection derived from an existing list type and implement **INotifyCollectionChanged** yourself, but it's much easier to use the **ObservableCollection<T>** list type that is already available to WinUI developers. This is a collection that will update the View when items are added or removed, or when the contents are completely refreshed.

The **Items** property of an **ObservableCollection<T>** is read-only, thus it cannot be directly set. You can add items by passing a **List<T>** or **IEnumerable<T>** to the constructor when creating **ObservableCollection<T>** or by using its **Add** or **Insert** methods (there is no **AddRange** method to add multiple items). You can update individual values in the collection by assigning a new item to the current index. You can remove items with the **Remove**, **RemoveAt**, **ClearItems**, or **Clear** methods.

In the next section, when we implement the MVVM pattern for ourselves, you will see these concepts in practice.

Implementing MVVM in WinUI applications

It's time to start converting our project to use MVVM. To gain a thorough understanding of the MVVM pattern, we will start by building our own MVVM infrastructure. For simple applications, it doesn't require more than a single base class:

1. Start by adding a **ViewModels** folder to the project. If you are using the code from GitHub, you can either continue with your project from the previous chapter or use the **Start** project in the folder for this chapter.
2. Next, add a new class to the **ViewModels** folder and name it **BindableBase**. This will be the base class for all our View Model classes in the project. It will be responsible for notifying the corresponding views of any property changes. This is accomplished by implementing the **INotifyPropertyChanged** interface. Let's review the **BindableBase** class code, as follows:

```
public class BindableBase : INotifyPropertyChanged
{
```

```
public event PropertyChangedEventHandler
    PropertyChanged;
protected void OnPropertyChanged(
    [CallerMemberName] string propertyName = null)
{
    PropertyChanged?.Invoke(this, new Property
        ChangedEventArgs(propertyName));
}
protected bool SetProperty<T>(ref T originalValue,
    T newValue, [CallerMemberName] string
        propertyName = null)
{
    if (Equals(originalValue, newValue))
    {
        return false;
    }
    originalValue = newValue;
    OnPropertyChanged(propertyName);
    return true;
}
}
```

By using this as the base class of our View Models, they will have two new methods available to use, as follows:

- **OnPropertyChanged:** Use this to trigger a **PropertyChanged** event to notify the View of changes in the data
- **SetProperty:** This method is used to set the value of a property if the value has been changed and will then call **OnPropertyChanged**

NOTE

*Be sure you have these two using directives at the top of the **BindableBase** class file:*

```
using System.ComponentModel;
```

```
using System.Runtime.CompilerServices;
```

Now that we have a base class, let's add our first View Model to the project. Right-click the **ViewModels** folder and add a new class named **MainViewModel**. This View Model is going to replace most of the code in the **MainWindow.xaml.cs** code-behind file for our **MainWindow**. The following code is part of the revised class. Please refer to **MainViewModel.txt** in the GitHub repository for the chapter (<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter03/Complete/MyMediaCollection/ViewModels>) for the current version of the complete class:

```
public class MainViewModel : BindableBase
{
    private string selectedMedium;
    private ObservableCollection<MediaItem> items;
    private ObservableCollection<MediaItem> allItems;
    private IList<string> mediums;
    public MainViewModel()
    {
        PopulateData();
    }
    ...
    public IList<string> Mediums
    {
        get
        {
            return mediums;
        }
        set
        {
            SetProperty(ref mediums, value);
        }
    }
    ...
}
```

You may have noticed that the code has been updated, using the new **BindableBase.SetProperty** method inside each property's **Set** block.

This ensures that the UI will be notified when the property value has been changed:

1. Now, we'll need to make the **MainViewModel** class available to the **MainWindow** view. Because there will be a single instance of this View Model used throughout the lifetime of the app, we will add a static read-only property to the **App.xaml.cs** file to make it available to the application, as follows:

```
public static MainViewModel ViewModel { get; } = new
MainViewModel();
```

2. We can now strip out all the code from **MainWindow.xaml.cs** that we copied to the **MainViewModel** class. In addition, add a property to make **App.ViewModel** available to **MainWindow** for data binding, as follows:

```
public sealed partial class MainWindow : Window
{
    public MainWindow()
    {
        this.InitializeComponent();
        Loaded += MainPage_Loaded;
    }
    public MainViewModel ViewModel => App.ViewModel;
    private async void AddButton_Click(object sender,
        Microsoft.UI.Xaml.RoutedEventArgs e)
    {
        var dialog = new MessageDialog("Adding items
            to the collection is not yet available.",
            "My Media Collection");
        await dialog.ShowAsync();
    }
}
```

The only other code we need to keep in the code-behind file right now is for the **Add** button, which we will update in the next section.

Finally, it's time to update the **MainWindow.xaml** file to bind to the data of the **MainViewModel**. There are only two changes required to handle the new data source, detailed next.

3. First, update **ComboBox** to remove the **x:Name** property and add **x:Bind** data binding for the **ItemsSource** and **SelectedItem** properties. The **SelectedItem** binding needs to be set to **TwoWay**. This will ensure that the data in **MainViewModel** is updated when the user changes **SelectedMedium** in the UI. The code can be seen in the following snippet:

```
<ComboBox ItemsSource="{x:Bind ViewModel.Mediums}"
  SelectedItem="{x:Bind ViewModel.SelectedMedium,
    Mode=TwoWay}" HorizontalAlignment="Right"
  MinWidth="120" Margin="0,2,6,4"/>
```

4. Now, update **ListView** to remove the **x:Name** property and add an **ItemsSource x:Bind** data binding, as follows:

```
<ListView Grid.Row="1" ItemsSource="{x:Bind
  ViewModel.Items}">
```

The assigned names for these controls are no longer needed because we are not referencing them anywhere in the code.

NOTE

*Assigning names to XAML elements allocates additional resources. It is recommended to only name elements when the elements must be referenced directly from code-behind files or by other View elements' data binding via **ElementName**.*

5. Now, run the application and try changing the **Medium** filter with the **ComboBox**. It should behave exactly as it did before, but now we have decoupled the View Model data from the UI, making it easier to test or potentially reuse.

Next, we will address the button **Click** event that we still have in the **MainWindow.xaml.cs** file.

Working with events and commands

It's time to update the project to move the event handling code to **MainViewModel**. By the end of this section, you will have removed all the code that was added to the **MainWindow.xaml.cs** file, except for the **ViewModel** property. This will be great for the separation of concerns, as well as for the maintainability and testability of the project.

We could use the same process of wiring up events with the **Add** button's **Click** event and connect it to a method on the **MainViewModel** class. There are two problems with this approach:

- The View and View Model layers become more tightly coupled, reducing maintainability
- UI concerns are injected into the view model, reducing the testability of the class

Let's take another route. The MVVM pattern has the concept of **Commands** to handle events. Instead of adding a handler to the event of our view element, we will bind that event to a property on the view model. The **Command** properties all expect a type of **System.Windows.Input.ICommand**.

Implementing ICommand

To use commands in the project, we'll start by creating an implementation of **ICommand**.

NOTE

*Another advantage of using an MVVM framework such as Prism or the MVVM Toolkit is that they provide implementations of **ICommand**.*

Add a new class to the **ViewModel** folder in the project and name it **RelayCommand**. This class will implement the **ICommand** interface. The **RelayCommand** class will look like this:

```
public class RelayCommand : ICommand
{
    private readonly Action action;
    private readonly Func<bool> canExecute;
    public RelayCommand(Action action)
        : this(action, null)
    {
    }
    public RelayCommand(Action action, Func<bool>
        canExecute)
    {
        if (action == null)
            throw new ArgumentNullException
                (nameof(action));
        this.action = action;
        this.canExecute = canExecute;
    }
    public bool CanExecute(object parameter) => canExecute
        == null || canExecute();
    public void Execute(object parameter) => action();
    public event EventHandler CanExecuteChanged;
    public void RaiseCanExecuteChanged() =>
        CanExecuteChanged?.Invoke(this, EventArgs.Empty);
}
```

RelayCommand has two constructors that both take an **Action** that will be invoked when the command is executed. One of the constructors also takes a **Func<bool>**. This will allow us to enable or disable UI actions based on the return value of **CanExecute**. We will use this to en-

able a **Delete** button only when a media item is selected in the **ListView**.

Using commands in the View Model

Now, it's time to update **MainViewModel** to handle commands from the **Add** and **Delete** buttons. In the next chapter, we will enhance the add operation so it can add or edit items. So, let's name the command and method accordingly:

1. First, add two new private variables to the **MainViewModel** class:

```
private MediaItem selectedMediaItem;  
private int additionalItemCount = 1;
```

The **additionalItemCount** variable is a temporary variable that we will use to track how many new items we have added to the list. The counter will help to generate unique IDs and names for each new media item. **selectedMediaItem** is a backing variable for the new **SelectedMediaItem** property.

2. Add the **SelectedMediaItem** property next:

```
public MediaItem SelectedMediaItem  
{  
    get => selectedMediaItem;  
    set  
    {  
        SetProperty(ref selectedMediaItem, value);  
        ((RelayCommand)DeleteCommand)  
            .RaiseCanExecuteChanged();  
    }  
}
```

In addition to calling **SetProperty** to notify the UI that **SelectedMediaItem** has changed, we also need to call **RaiseCanExecuteChanged** on a new **DeleteCommand**.

3. Next, let's add **DeleteCommand** and **AddEditCommand** along with their corresponding actions:

```
public ICommand AddEditCommand { get; set; }
public void AddOrEditItem()
{
    // Note this is temporary until
    // we use a real data source for items.
    const int startingItemCount = 3;
    var newItem = new MediaItem
    {
        Id = startingItemCount + additionalItemCount,
        Location = LocationType.InCollection,
        MediaType = ItemType.Music,
        MediumInfo = new Medium { Id = 1, MediaType =
            ItemType.Music, Name = "CD" },
        Name = $"CD {additionalItemCount}"
    };
    allItems.Add(newItem);
    Items.Add(newItem);
    additionalItemCount++;
}
public ICommand DeleteCommand { get; set; }
private void DeleteItem()
{
    allItems.Remove(SelectedMediaItem);
    Items.Remove(SelectedMediaItem);
}
private bool CanDeleteItem() => selectedMediaItem !=
    null;
```

There is an **ICommand** property for each UI operation (**AddEditCommand** and **Delete Command**) and new methods for each command to execute (**AddOrEditItem** and **DeleteItem**). There is also a **CanDeleteItem** method that returns a **bool** value to indicate whether a media item has been selected by the user.

4. At the end of the **MainViewModel** constructor, add two lines of code to initialize the commands, connecting them to the corresponding

methods:

```
DeleteCommand = new RelayCommand>DeleteItem,
    CanDeleteItem);
// No CanExecute param is needed for this command
// because you can always add or edit items.
AddEditCommand = new RelayCommand>AddOrEditItem);
```

The preceding code is part of the revised **MainViewModel** class. Please refer to **MainViewModel12.txt** in the GitHub repository for the chapter (<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/main/Chapter03/Complete/MyMediaCollection/ViewModels>) for the current version of the complete class.

Our View Model has been updated to use commands. Next, we will update the view to bind to them.

Updating the View

Our view model is ready to go. It's now safe to remove all the event handling code from the **MainWindow** code. It should look like this when you're finished:

```
public sealed partial class MainWindow : Window
{
    public MainWindow()
    {
        this.InitializeComponent();
    }
    public MainViewModel ViewModel => App.ViewModel;
}
```

The **MainWindow.xaml** file will need a few updates to get the **Add Item** and **Delete Item** buttons fully functional with the temporary test data. Perform the following steps to accomplish this:

1. Update the **ListView** to bind the **SelectedItem** property to **SelectedMediaItem** in the view model:

```
<ListView Grid.Row="1" ItemsSource="{x:Bind
  ViewModel.Items}" SelectedItem="{x:Bind
    ViewModel.SelectedMediaItem, Mode=TwoWay}">
```

TwoWay data binding is required to allow the UI to update the view model.

2. Next, move the **Add Item** button inside the first column of a new two-column **Grid**. Then, create a **Delete Item** button in the second column. Remove the **Click** event handler and set the properties of the grid and two buttons to match the following snippet:

```
<StackPanel Grid.Row="2"
  HorizontalAlignment="Right"
  Orientation="Horizontal">
  <Button Command="{x:Bind
    ViewModel.AddEditCommand}"
    Content="Add Item"
    Margin="8,8,0,8"/>
  <Button Command="{x:Bind ViewModel.DeleteCommand}"
    Content="Delete Item"
    Margin="8"/>
</StackPanel>
```

Each button's **Command** property will be bound to the new **ICommand** properties in the view model. The **Command** property of a button is invoked when it's clicked by the user.

3. We're now done updating the project to use MVVM. Run the application to see how it works.
4. When it first loads, the **Delete Item** button will be disabled. Select an item in the list and notice that the button is automatically enabled. If you click **Delete Item**, the selected item is removed from the list and the button is disabled again.
5. Finally, click **Add Item** a few times to see how new items are created and added to the list. Each new item has a unique name using

the counter we created in the view model, as illustrated in the following screenshot:

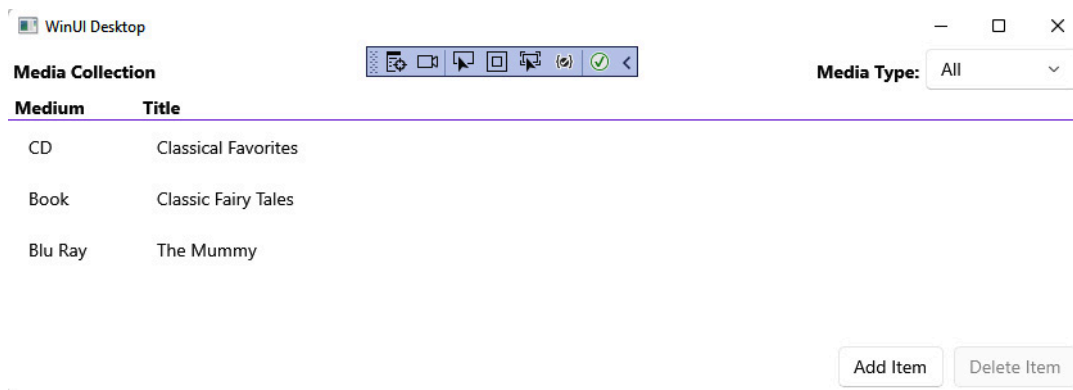


Figure 3.2 – My Media Collection after adding and removing a few items

MainWindow now has a view model that is completely decoupled from any UI concerns. This will allow you to maximize the unit test coverage on the project.

Before we wrap up, let's see how we can reduce the amount of code in our project by using an MVVM framework such as the MVVM Toolkit.

Leveraging the MVVM Toolkit

We briefly introduced the MVVM Toolkit earlier in the chapter. In this section, we will update the **MainViewModel** to see how we can remove the need for a **BindableBase** class and reduce the amount of code in the view model itself:

1. Start by right-clicking on the solution file in **Solution Explorer** and select **Manage NuGet Packages for Solution**.
2. In the NuGet window, select the **Browse** tab and search for **CommunityToolkit.Mvvm**.
3. Select the **CommunityToolkit.Mvvm** package in the results and install the latest stable version (8.2.0 or later).

4. Close the NuGet window and open the **MainViewModel** class. The first thing we need to do to use the MVVM Toolkit's source generators is update **MainViewModel** to be a partial class and have it inherit from **CommunityToolkit.Mvvm.ComponentModel.ObservableObject** instead of our own **BindableBase** class:

```
public partial class MainViewModel : ObservableObject
```

NOTE

To learn more about how the MVVM Toolkit uses .NET source generators to generate public properties and commands, check out the documentation on Microsoft Learn:

<https://learn.microsoft.com/dotnet/communitytoolkit/mvvm/generators/overview>. If you are unfamiliar with .NET source generators, you can also read about them on Microsoft Learn in the .NET documentation: <https://learn.microsoft.com/dotnet/csharp/roslyn-sdk/source-generators-overview>.

5. Next, each simple property that only calls **SetProperty** in its setter will be removed, and the backing variable will be updated to have an **ObservableProperty** attribute. Remove the **Mediums** and **Items** properties and the private **mediums** and **items** fields should now look like this:

```
[ObservableProperty]  
private IList<string> mediums;  
[ObservableProperty]  
private ObservableCollection<MediaItem> items;
```

We have two public properties remaining in the class. Let's start with **SelectedMedium**. We will remove **SelectedMediaItem** after updating the commands.

6. The setter for **SelectedMedium** has some custom logic after the call to **SetProperty**. Create a new partial method named **OnSelectedMediumChanged** to contain that code:

```
partial void OnSelectedMediumChanged(string value)
{
    Items.Clear();
    foreach (var item in allItems)
    {
        if (string.IsNullOrEmpty(value) ||
            value == "All" ||
            value == item.MediaType.ToString())
        {
            Items.Add(item);
        }
    }
}
```

Notice that we also updated the uses of **selectedMedium** to **value**. The **value** parameter contains the new **selectedMedium** value. You may have also noticed some green squiggles in Visual Studio inside the **PopulateData** method, indicating that you should be using the generated properties instead of using the private **ObservableProperty** members directly. You can update all of these uses by capitalizing the first letter of each variable.

7. Next, we will update the two commands. Add the **RelayCommand** attribute to the **AddOr EditItem** and **DeleteItem** methods and rename them to **AddEdit** and **Delete** to ensure that the generated commands have the same names that our old commands had.
8. Remove the two commands and the code to initialize them in the **MainViewModel** constructor. The code for the constructor and the signatures for the two command methods should now look like this:

```
public MainViewModel()
{
```

```

        PopulateData();
    }
    [RelayCommand]
    public void AddEdit()
    {
        ...
    }
    [RelayCommand(CanExecute = nameof(CanDeleteItem))]
    public void Delete()
    {
        ...
    }
    private bool CanDeleteItem() => SelectedMediaItem !=
        null;

```

The attribute for **DeleteItem** also indicates that **CanDeleteItem** should be used for checking whether the command can be invoked.

9. Finally, let's replace the **SelectedMediaItem** property with a source-generated property from the MVVM Toolkit. The existing public property tells the **DeleteCommand** that **CanExecute** should be checked. Remove that property and update the private **selected-MediaItem** variable to add two attributes:

```

[ObservableProperty]
[NotifyCanExecuteChangedFor(nameof>DeleteCommand))]
private MediaItem selectedMediaItem;

```

In addition to the **ObservableProperty** attribute, we've added a **NotifyCan ExecuteChangedFor** attribute, providing the name of the **DeleteCommand** generated **ICommand** property. The generated property, which is invisible to us, would be equivalent to this:

```

public MediaItem SelectedMediaItem
{
    get => selectedMediaItem;
    set
    {
        if (SetProperty(ref selectedMediaItem, value))
        {

```

```
        DeleteCommand.NotifyCanExecuteChanged();  
    }  
}  
}
```

That's all you need to do. There are no changes needed in the view, and the application works exactly as it did before. All we've done is remove quite a bit of code from **MainViewModel**.

We will continue to use the MVVM Toolkit throughout the remaining chapters. Now, let's review what we've learned in this chapter.

Summary

We've made quite a bit of progress with the application in this chapter. While it's not yet connected to a live data source, we have methods in place to add and remove items from the media collection in memory. In addition, the project has been refactored to use the **MVVM** pattern, moving all the existing view logic from the **MainWindow** code-behind file to a new **MainViewModel** class. The new **MainViewModel** class has no dependencies on the UI. Finally, we saw how integrating the MVVM Toolkit into the project can reduce the boilerplate code in our view models. These good software design habits will serve us well in the chapters ahead as we build more functionality onto the project.

In the next chapter, we will continue learning how to use the MVVM pattern to write robust, maintainable WinUI applications. We will cover some more advanced MVVM topics and learn some techniques for window management in a WinUI project.

Questions

1. What does MVVM stand for?

2. Which layer typically defines the business entities in the MVVM pattern?
3. Name one of the popular MVVM frameworks discussed in the chapter.
4. Which interface must every View Model class implement in an MVVM application?
5. Which special collection type in .NET notifies the UI of changes to the collection, via data binding?
6. Which control property of **ComboBox** and **ListView** is used to get or set the currently selected item in the control?
7. Which interface is implemented to create commands for event binding?